

Análise de Complexidade de Algoritmos

Objetivos

Objetivo Principal

- Avaliar a eficiência de um algoritmo no uso de recursos tais como memória e processamento

Outros objetivos

- Compreender o que significa medir o custo do algoritmo
- Estabelecer a função de complexidade de um algoritmo
- Realizar uma análise de melhor caso, pior caso e do caso médio
- Entender a motivação para se estudar comportamento assintótico
- Compreender as notações voltadas para o estudo do comportamento assintótico de funções
- Aplicar técnicas para análise de complexidade

Roteiro

Introdução

Medida de Custo do Algoritmo

Abordagem Matemática e Função de Complexidade

Caso Médio, Pior Caso e Melhor Caso

Comportamento Assintótico

Notações

Classes de Comportamento Assintótico

Técnicas de Análise de Complexidade

Projeto vs. Análise

Após a conclusão do projeto do algoritmo, uma análise deve ser feita quanto a viabilidade de uso do algoritmo

Algumas questões importantes

- Correção do Algoritmo (Estar correto)
 - Atender à condição de parada
 - Apresentar a solução correta
- Eficiência do algoritmo (Ser eficiente)

Motivação

- Conforme Knuth, existem dois enfoques na análise de algoritmos:
 - Analisar um algoritmo em específico
 - Qual o custo de usar um algoritmo em específico?
 - Qual o número de instruções necessárias e qual o uso de memória?
 - Analisar uma classe de algoritmos
 - Para um determinado problema, a ordenação por exemplo, qual o melhor algoritmo a ser considerado?

Motivação

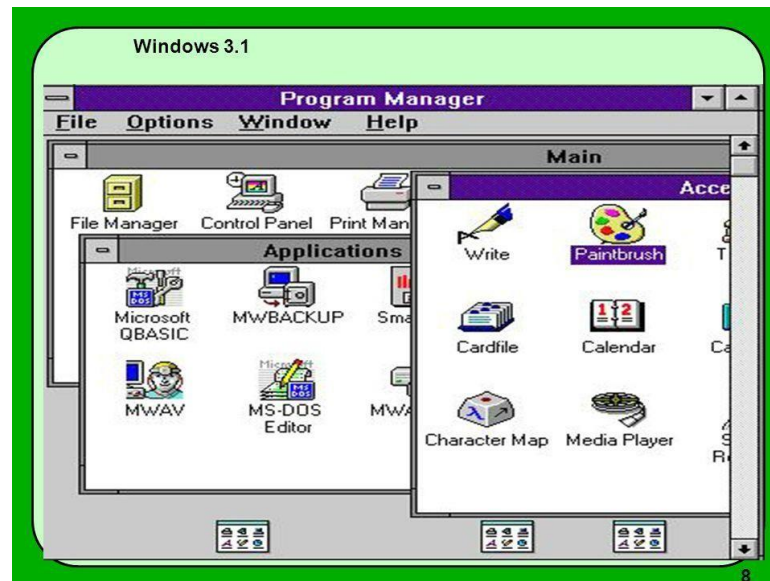
- Cenário
 - Ao realizar uma consulta em banco de dados, é comum fazer uma análise de como o banco de dados vai processar a consulta
 - Isto é, o Plano de Execução da consulta
- Curiosidade
 - Em 1960, a grande dificuldade da IA foi estabelecer algoritmos que pudessem ser executados em tempo viável

Roteiro

- Introdução
- **Medida de Custo do Algoritmo**
- Abordagem Matemática e Função de Complexidade
- Caso Médio, Pior Caso e Melhor Caso
- Comportamento Assintótico
- Notações
- Classes de Comportamento Assintótico
- Técnicas de Análise de Complexidade

O que medir?

- Medir o custo do algoritmo, significa medir os seguintes recursos usados
 - Memória
 - Tempo de Execução
 - Utilização de outros recursos
 - rede
 - consumo de energia



Foco

- A princípio, nosso principal foco na análise de algoritmos será o **tempo de execução**
- Quando necessário analisar outros fatores: uso de memória por exemplo, indicaremos em específico
- Questionamento
 - Como medir o tempo de execução de um algoritmo?

Formas de Medição

- Execução em um computador
 - O algoritmo é executado em um computador e o tempo de execução é medido diretamente
- Análise matemática
 - Um modelo matemático é criado sobre o algoritmo e daí tem-se uma análise do tempo de execução

Vantagens e desvantagens

- Na medição direta do algoritmo, através de sua execução em um computador, existem as seguintes desvantagens:
 - Dependência do compilador
 - Dependência do hardware
 - Uso de grande memória que pode impactar indiretamente na performance
- Aplicado em alguns cenários, quando se tem um mesmo compilador e hardware e deseja-se testar um conjunto de algoritmos
- Ideia chave:
 - Padronizar os critérios para a comparação
 - Labs de revistas de informática

Roteiro

- Introdução
- Medida de Custo do Algoritmo
- **Abordagem Matemática e Função de Complexidade**
- Caso Médio, Pior Caso e Melhor Caso
- Comportamento Assintótico
- Notações
- Classes de Comportamento Assintótico
- Técnicas de Análise de Complexidade

Modelo Matemático

- O método mais consistente para avaliar o custo do algoritmo é através do Modelo Matemático
- A ideia é encontrar um modelo que considera o número de instruções executadas pelo algoritmo
- Este número está diretamente relacionado com o tempo de execução do algoritmo

Iniciando

- Implemente uma função que retorna o último nó de uma lista encadeada que contém uma determinada chave (key).

public int ultimo (Linklist L, int key)

- Avalie o custo do algoritmo levando em conta o número de instruções e o custo de cada instrução
- Considere:
 - Comparação: Custo 1
 - Atribuição: Custo 2
 - Oper. Matemática: Custo 3

```
public int ultimo (Linklist L, int key)
```

```
    Link c = L.Getfirst ( );  
    while (c != null){  
        if (c.Getnext ( ) == null){  
            c.SetiData (key);  
        }  
        c = c.Getnext ( );  
    }  
    return key;
```

```
Class Link {  
    Public int iData;  
    Public Link next;  
  
    Public Link (int ID) {  
        iData = ID;  
    }  
}
```

```
Class LinkedList {  
    Private Link First;  
    Public LinkedList ( ){  
        First = null;  
    }  
}
```

```
Class void insertFirst (int ID) {  
    Link NL = New Link (ID);  
    NL.next = first;  
    first = NL;  
}
```

Conclusão do Exercício

Passo 1:

- Resolver o problema

Passo 2:

- Caracterizar a entrada

Passo 3:

- Encontrar a relação entre o número de entrada e o número de instruções
- Encontrar a função de complexidade que estabelece esta relação entre número de instruções e tamanho da entrada

Exemplo

(1) pos = -1

(1) elem = 0

```
          (1)  (n)  (n)
          for (i=0; i < n; i++){
(n)      if (v [ i ] == elem){
(1)      pos = i;
          }
        }
```

(1) return (pos)

1 + 1 + 1 + n + n + n + 1 + 1

3n + 5



Função de complexidade

Conclusão do Exercício

- Já foi dado o primeiro passo na Análise do Algoritmo
 - Foi feita uma contagem (abordagem matemática) do custo (número de instruções)
 - Veja que isso independe de onde (qual máquina) o algoritmo vai rodar
- Análise do Algoritmo
 - É avaliar o custo do algoritmo
 - Custo está sendo relacionado com o tempo de execução
 - O tempo de execução é proporcional ao número de instruções executadas e respectivos custos

Criando um modelo matemático

- O que é um modelo?
 - É uma abstração sobre algo, que foca o que de fato importa para um determinado problema
 - Modelo Físico : Leis de Newton
 - Modelo de Sistemas de Informação
- No nosso caso,
 - a simplificação da abordagem empregada no exercício anterior
 - visa concentrar a Análise ao que de fato é relevante
 - para uma avaliação do desempenho do algoritmo

Questionamento

- É necessário classificar as instruções em tipos e atribuir um peso a cada uma?
- De todas as instruções, quais as que ocorrem com maior frequência e quais as que ocorrem com menor frequência?

Simplificando

- Vale simplificar na Análise do Algoritmo
 - Todas as instruções tem um mesmo peso
 - Para um determinado problema, vale considerar a operação predominante
 - Por exemplo
 - na ordenação, o número de comparações é considerado, pois este número está no “núcleo” do algoritmo
- Isso é baseado no computador MIX idealizado por Knuth (1968)

Relação com a Entrada

- Relembrando:
 - O que são Instâncias do Problema?
- O número de instruções do algoritmo pode variar conforme cada instância do problema, por isso é necessário compreender algumas características da entrada
- É possível caracterizar o tamanho da instância do problema através de uma variável “n”, comumente chamado de Tamanho da Entrada

Características da Entrada

- Exemplos
 - Para um cálculo fatorial, o tamanho da entrada é o próprio valor de entrada da função
 - Para uma busca ou ordenação em um vetor, o tamanho da entrada é o número de casas do vetor
 - Para um processamento de uma árvore, o tamanho da entrada pode ser os níveis da árvore ou o número de nós da árvore
 - Para uma busca em grafo, o número da entrada pode ser o número de nós do grafo

Função de Complexidade

- A função de complexidade determina a relação entre o tamanho da entrada e o custo do algoritmo
- No nosso caso, o número de instruções ou medida de tempo de execução do algoritmo

Exercício

- Estabeleça a função de complexidade do algoritmo do exercício anterior, de busca do maior elemento em um vetor.